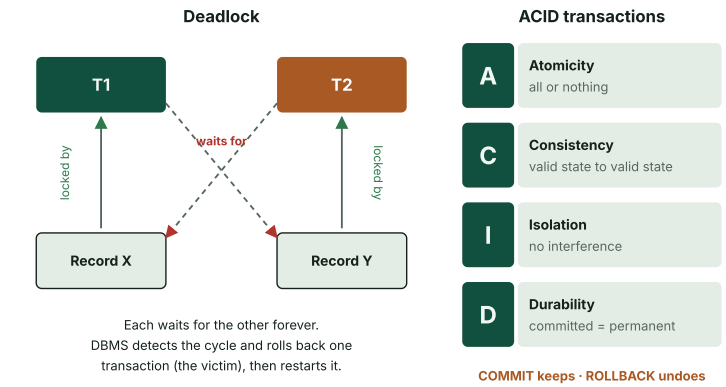
Multi-user databases: transactions, ACID, locking and distributed databases

IS07 FACT FILE

Specification: students should be able to

- Explain transactions and the ACID properties
- Describe record locking, deadlock and how deadlock is handled
- Explain commit and rollback
- Compare client–server and distributed databases

Key words: transaction, ACID, atomicity, consistency, isolation, durability, commit, rollback, record locking, deadlock, lost update, client–server, distributed database



Deadlock: transaction T1 holds a lock on record X and waits for Y while T2 holds Y and waits for X; the DBMS rolls one back. ACID properties listed alongside

Transactions and ACID

A transaction is a sequence of operations treated as a single unit. The ACID properties make transactions reliable. Atomicity means all or none of the operations take effect. Consistency means a transaction takes the database from one valid state to another, obeying all integrity rules. Isolation means concurrent transactions do not interfere and give the same result as running one after another. Durability means committed changes are permanent, even after a crash, usually because they are written to a transaction log.

Concurrency control

When several users update the same data at once, the lost update problem can occur: one user's change overwrites another's. Record locking prevents this by giving a transaction exclusive access to a record until it commits or rolls back. Locking can lead to deadlock, where two transactions each hold a lock the other needs. A DBMS detects deadlock and rolls back one transaction, or prevents it by ordering locks or requesting all locks at the start.

Client–server and distributed

In a client–server database the data and DBMS are held centrally and clients send queries, receiving only the results; this gives central control and lower network traffic but creates a single point of failure. A distributed database spreads one logical database across several sites using fragmentation and replication. It gives fast local access and resilience, but keeping replicas consistent, securing several sites and committing transactions across sites is complex.

T Key terms from the mark schemes

Transaction
A sequence of database operations treated as a single logical unit that must either complete entirely or not at all.

ACID
Atomicity, consistency, isolation and durability: the properties that guarantee database transactions are processed reliably.

Record locking
A concurrency control technique that prevents other users updating a record while one transaction is using it.

Deadlock
A situation where two or more transactions each hold a lock the other needs, so none can proceed.

Rollback
Undoing all changes made by a transaction since it began, returning the database to its previous consistent state.

Distributed database
A single logical database whose data is stored at several sites linked by a network but appears to users as one database.

Exam focus

Each ACID property needs a definition plus an applied example to gain full marks; use the scenario given. For locking questions describe the sequence of events with times or users (A reads, B reads, A writes, B writes) – timelines earn marks. Deadlock answers need the cause (each holds a lock the other wants) and the resolution (DBMS detects, rolls back one). For 'discuss' questions on distributed databases balance local speed and resilience against complexity, security and consistency, and conclude.

? Check yourself

1. Define the term transaction in the context of databases. [2]
2. Explain atomicity and durability, using a bank transfer as an example. [4]
3. Describe how the lost update problem can occur and how record locking prevents it. [4]

Answers: 1. A transaction is a group of one or more database operations (e.g. updates) (1) / that must be treated as a single logical unit of work: either all succeed (COMMIT) or none are kept (ROLLBACK) (1). 2. Atomicity: all operations in the transaction happen or none do (1) / e.g. the £50 is never debited from savings without also being credited to the current account; if the credit fails the debit is rolled back (1) / Durability: once the transaction is committed its changes are permanent (1) / e.g. the transfer is still recorded after a power cut or server crash/restart, because it was written to the log/disk (1). 3. Two users / transactions read the same record at the same time, e.g. both read Balance = 100 (1) / each changes their copy and writes it back, e.g. A writes 70 and then B writes 80, so A's update is overwritten (lost); the balance should be 50 (1) / With record locking, the first transaction to read the record for update locks it (exclusive lock) (1) / the second must wait until the lock is released at COMMIT/ROLLBACK, so it reads the updated value (70) and writes 50 (1).